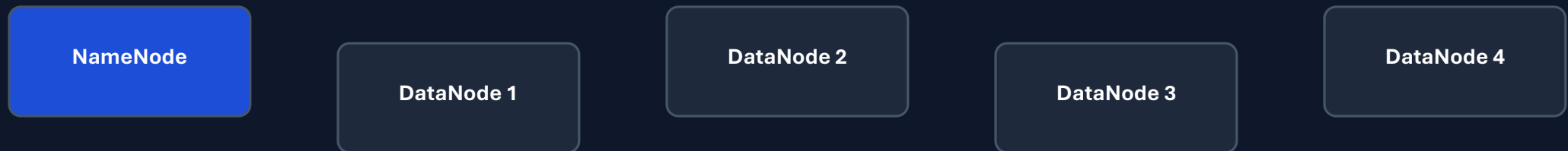


HDFS - Hadoop Distributed File System

How Hadoop stores massive datasets across multiple machines



Remember Our Problem?

Conserver plusieurs années de journaux techniques pour le projet DORA

1 · PRODUCE

Payment API
Authentication
Core Banking
Infrastructure

2 · GROW

2 TB / day
≈ 730 TB / year
Multi-year retention

3 · RETAIN

Durable
Scalable
Auditable
Cost-aware

Question : peut-on raisonnablement conserver tout cela sur un seul serveur ?

Can One Server Store Everything?

ONE SERVER

CPU · RAM · DISKS

Capacity limit

Single point of failure

Expensive scale-up

Maintenance impact

Le problème n'est pas seulement la capacité : c'est aussi la disponibilité et l'évolutivité.

What Do We Need?

Scale-Out

Add machines, not only bigger machines

Fault Tolerance

Failures must be expected

High Throughput

Stream very large datasets

Large Files

Optimized for big sequential I/O

HDFS in One Sentence

ONE FILESYSTEM

HDFS

MANY MACHINES

NameNode

DataNode 1

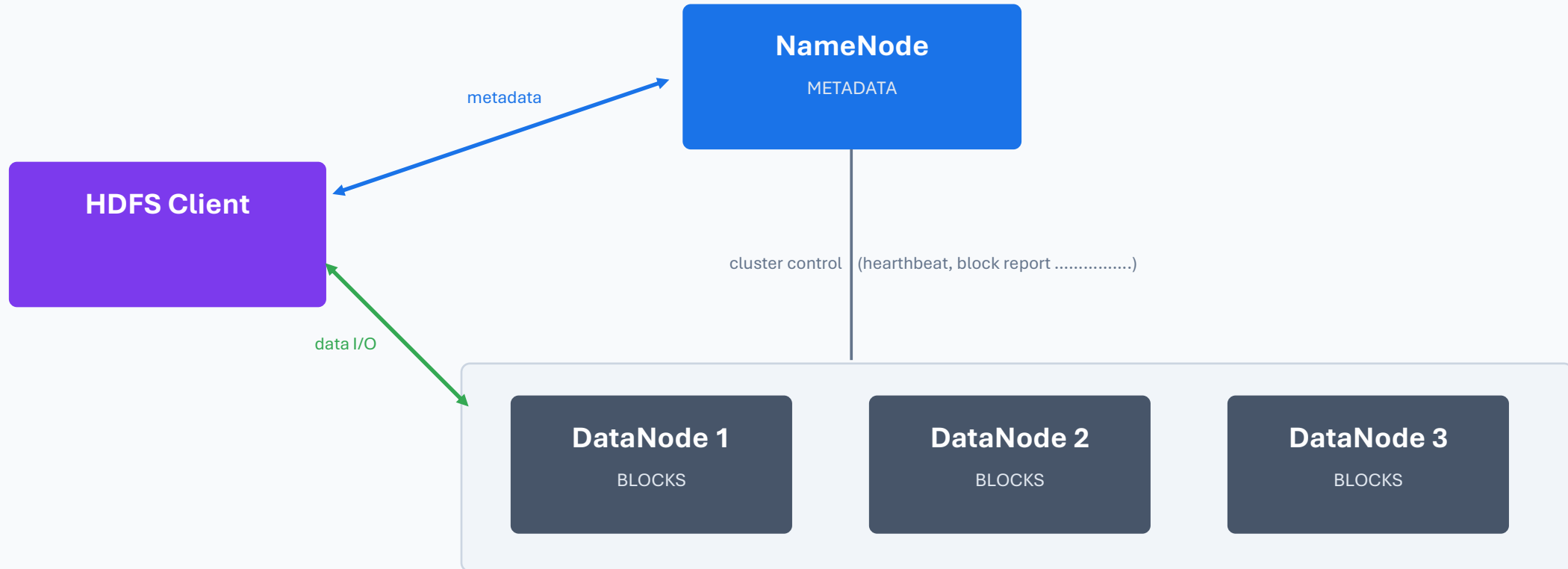
DataNode 2

DataNode 3

HDFS présente un espace logique unique, alors que les données sont distribuées sur plusieurs machines.

HDFS Architecture

Deux chemins : métadonnées et données



Le client échange les métadonnées avec le NameNode et les data directement avec les DataNodes.

The NameNode - The Brain of HDFS

NameNode

Metadata center

Namespace

Files, directories, permissions

Block mapping

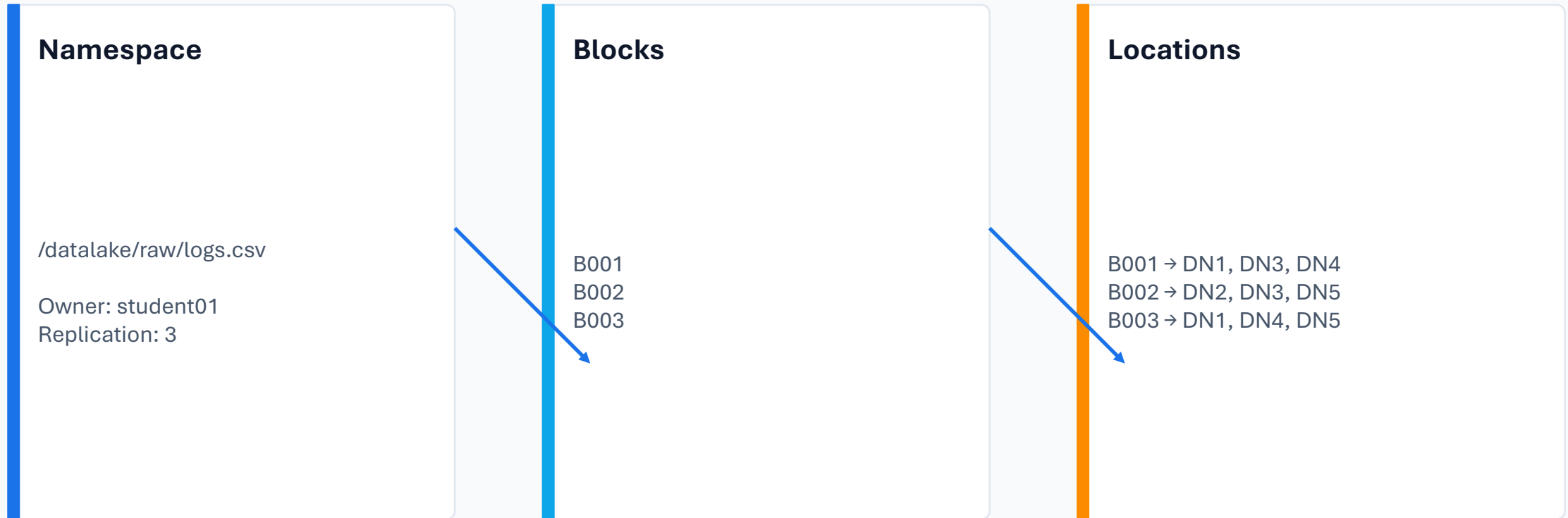
Which blocks belong to which file?

Locations

Which DataNodes currently hold each block?

Le NameNode sait où sont les données ; il ne contient pas les données utilisateur.

What Does the NameNode Know?



Le TP utilisera fsck pour rendre cette relation fichier → blocs → DataNodes visible.

The DataNodes - Where Data Lives



DataNodes : lecture/écriture des blocs, création/suppression/réplication sur instruction du NameNode.

NameNode vs DataNode

NameNode

METADATA

- Namespace
- Block map
- Replication decisions
- Cluster metadata

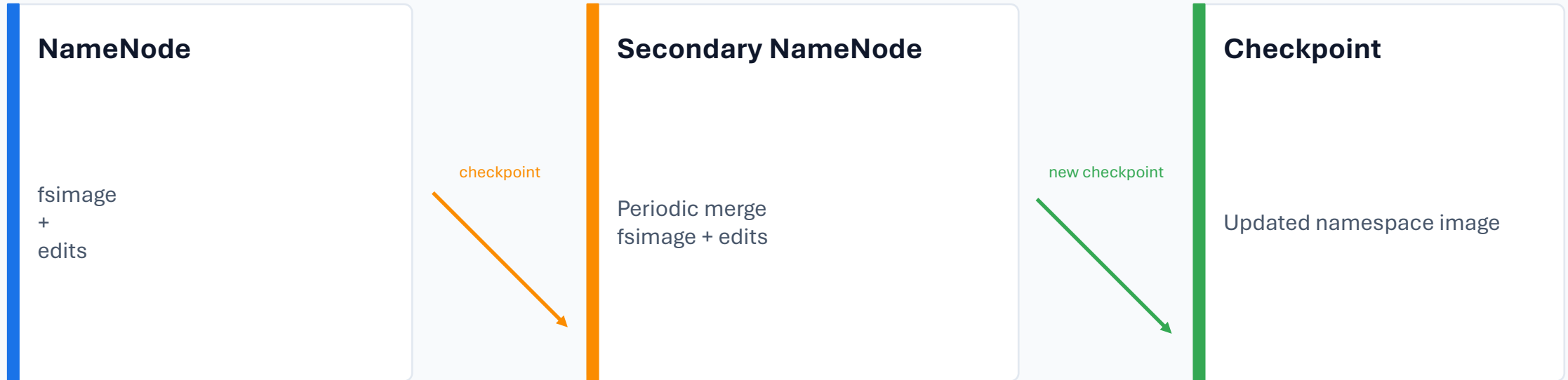
DataNodes

ACTUAL DATA

- HDFS blocks
- Read / write
- Block reports
- Replication execution

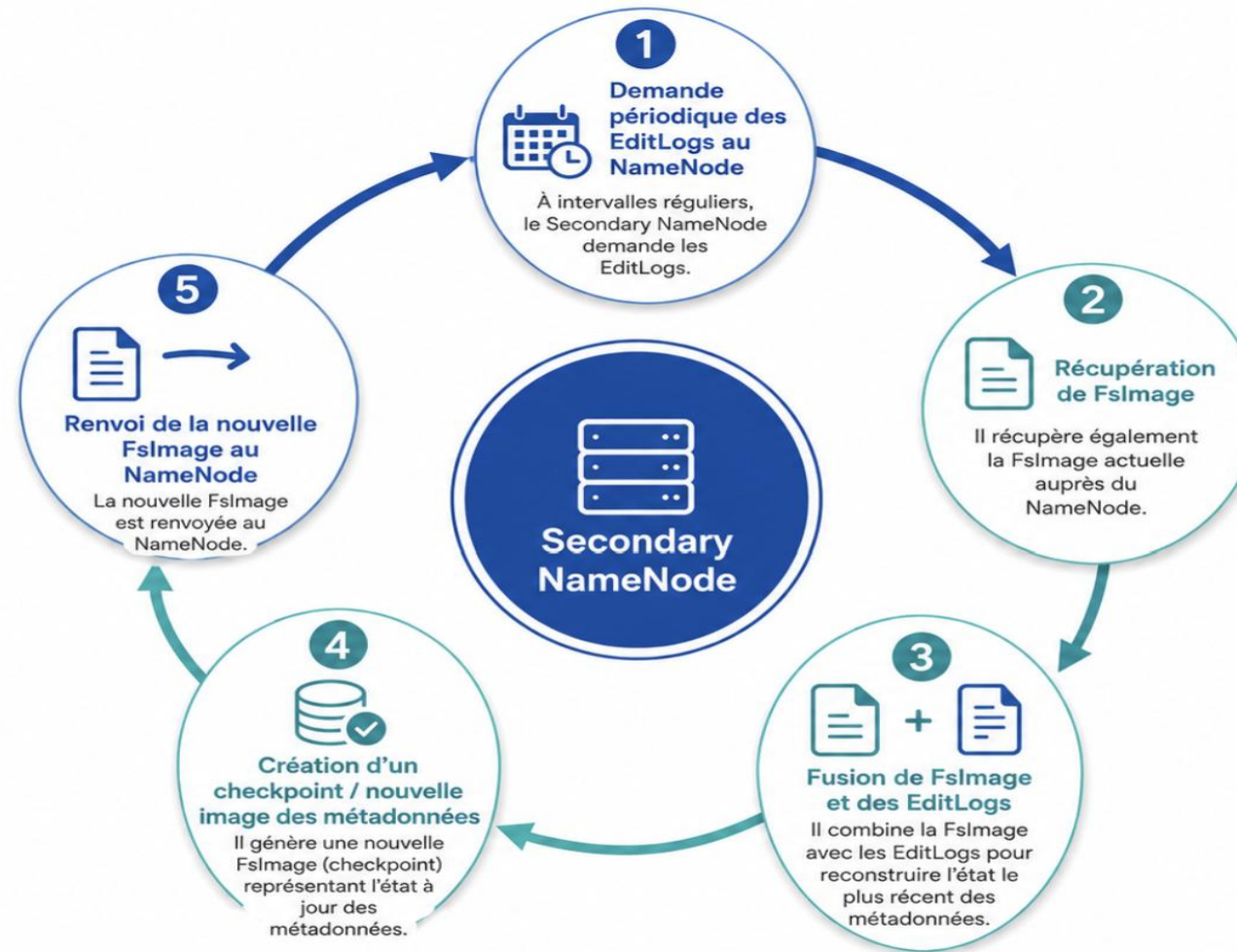
À retenir : Metadata ≠ Data.

Secondary NameNode ≠ Backup NameNode



Le Secondary NameNode fait des checkpoints ; ce n'est pas un Standby NameNode.

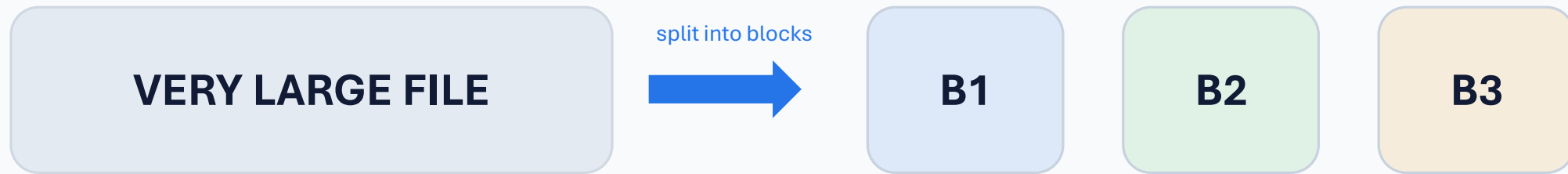
Secondary NameNode ≠ Backup NameNode



Le Secondary NameNode n'est pas un nœud de secours ; il sert à créer des checkpoints des métadonnées.

Why Does HDFS Use Blocks?

A file can be larger than a single disk, and different blocks can be stored independently.

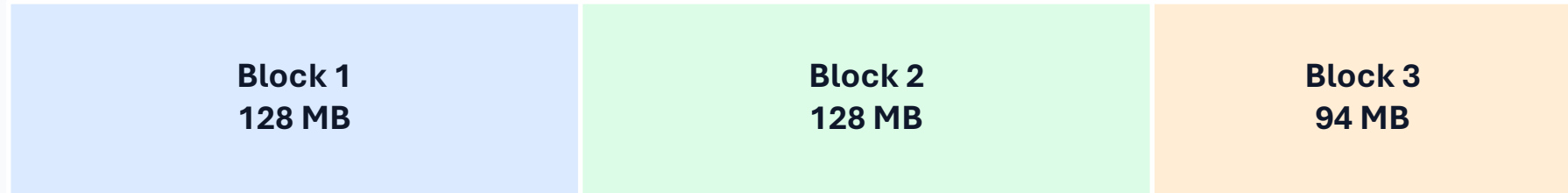


Les blocs permettent de stocker et de traiter un fichier très volumineux sur plusieurs machines.

One File → Multiple Blocks

Exemple avec une taille de bloc de 128 MB

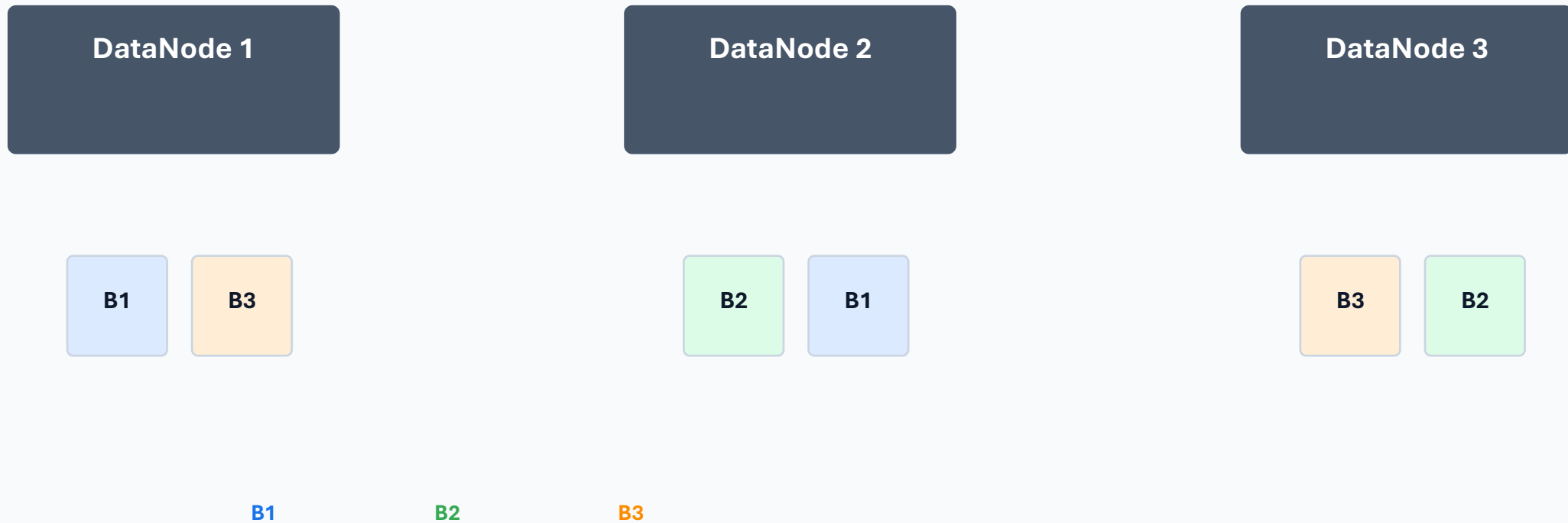
350 MB



Important

Le dernier bloc n'occupe que la taille réellement nécessaire. Un petit fichier ne consomme pas automatiquement 128 MB.

Blocks Across the Cluster



Un fichier HDFS n'est pas "sur une machine" : ses blocs sont distribués dans le cluster.

What If a DataNode Dies?

DataNode 1

B1 · B3

DataNode 2

FAILED

DataNode 3

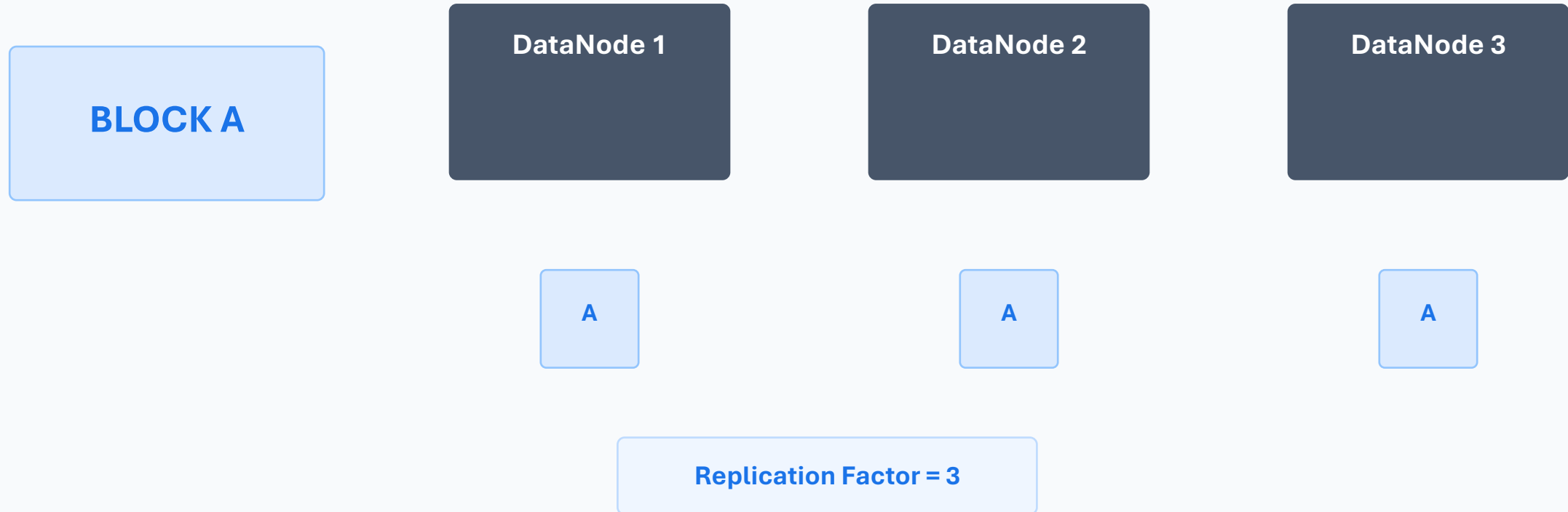
B2 · B3



Question : si une seule copie de B2 était sur DataNode 2, que deviendrait le fichier ?

Replication

Fault tolerance by design



Replication Factor = nombre de copies maintenues pour chaque bloc du fichier.

Replication Factor

RF = 1

Capacity efficient

Low resilience

RF = 2

More resilience

2× data copies

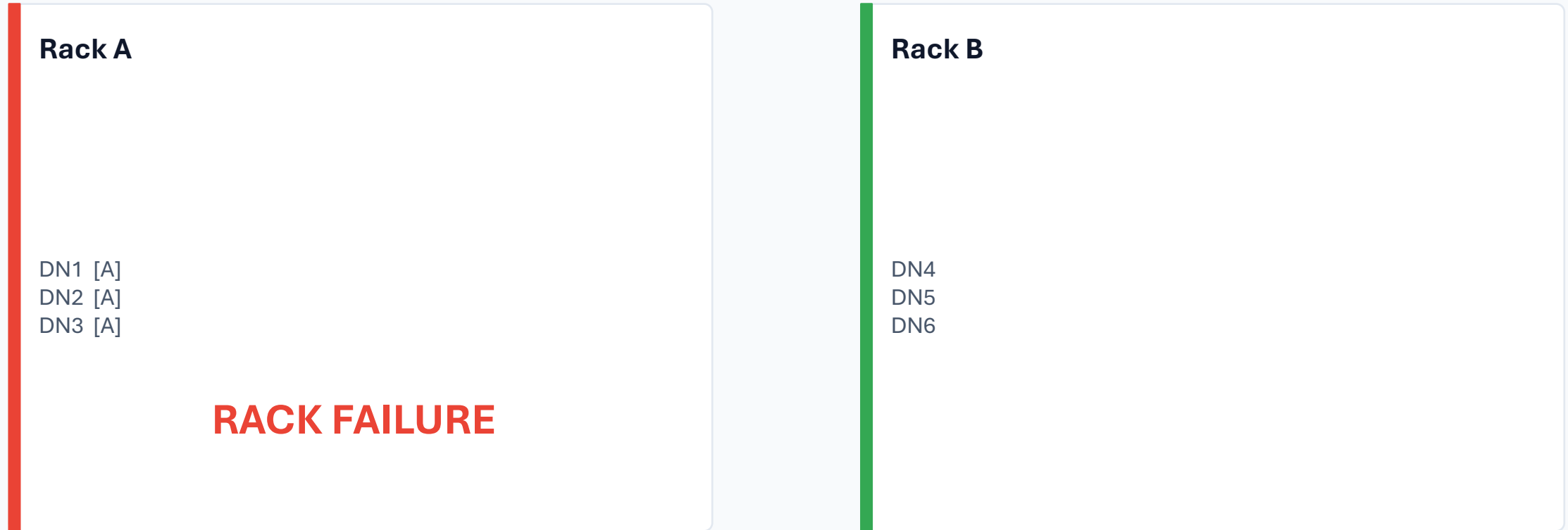
RF = 3

Recommended

3× data copies

Plus de réplication = plus de disponibilité potentielle, mais aussi plus de capacité consommée.

But What If an Entire Rack Fails?



Trois copies dans le même rack ne protègent pas contre la perte du rack.

Rack Awareness

HDFS understands network topology

NameNode

stores rack topology

Topology metadata: DN1, DN2, DN3 → /A · DN4, DN5, DN6 → /B

Rack /A

DN1

DN2

DN3

Rack /B

DN4

DN5

DN6

Le NameNode utilise l'appartenance aux racks dans ses décisions de placement.

Replica Placement Strategy

Exemple classique pour RF = 3

Rack A

DN1 [A]
DN2
DN3

1 replica

Rack B

DN4 [A]
DN5 [A]
DN6

2 replicas

Objectif : survivre à une panne de rack sans multiplier inutilement le trafic inter-rack.

Why Rack Awareness Matters

Fault Tolerance

Node failure ✓
Rack failure ✓
Copies separated physically

Network Locality

Same rack → cheaper
Cross-rack → more network cost
Read closest replica when possible

Rack Awareness = résilience + optimisation du trafic réseau.

Reliability Has a Cost

1 TB

× RF 3 =

1 TB

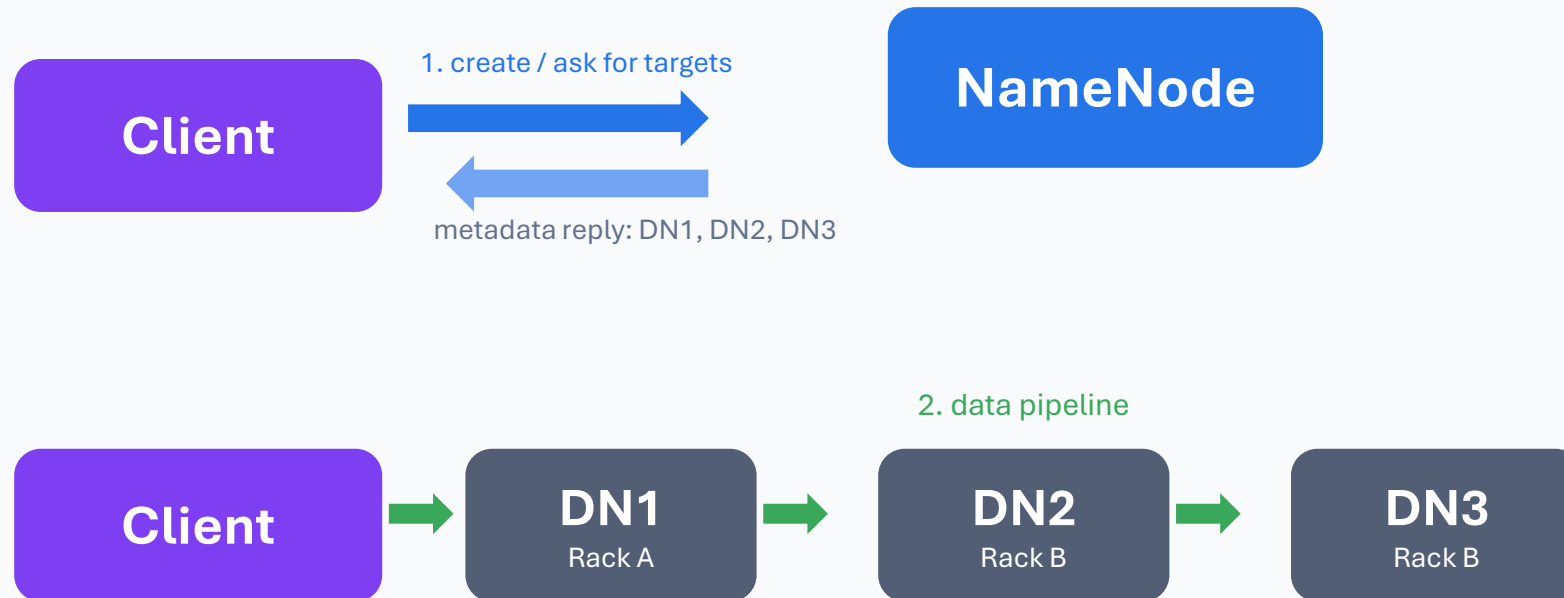
1 TB

1 TB

Trade-off

≈ 3 TB de données répliquées pour 1 TB logique avec RF=3 - avant autres surcoûts et réserves.

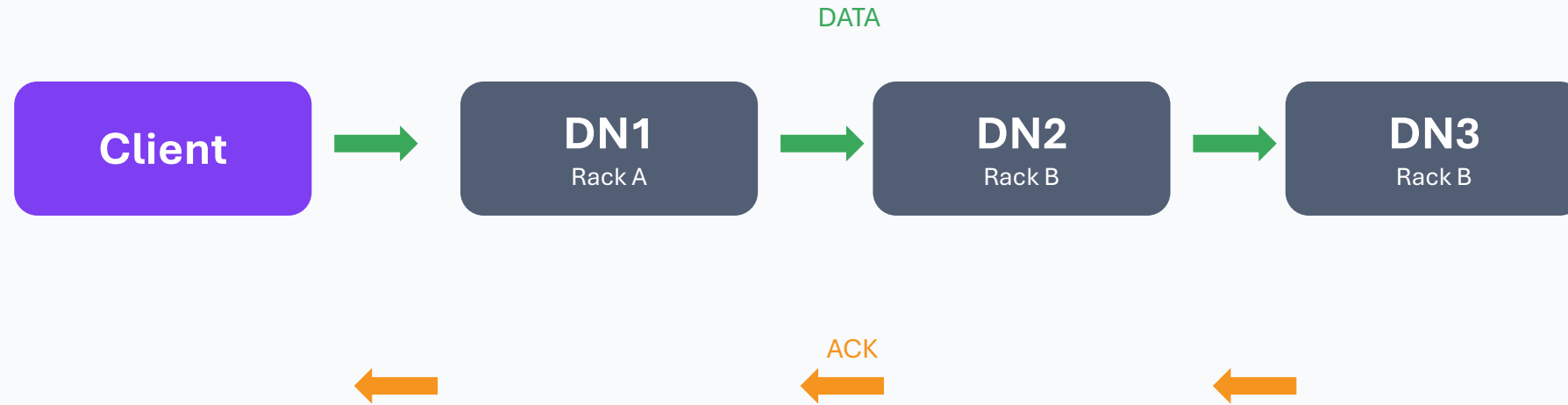
How HDFS Writes a File



1) échange de métadonnées avec le NameNode ; 2) le client envoie ensuite les données au pipeline de DataNodes.

The Write Pipeline

Two directions, two meanings



Les données circulent de gauche à droite ; les acquittements reviennent de droite à gauche.

Metadata Path ≠ Data Path

Metadata Path

Client → NameNode

Open / create

Block locations

Namespace operations

Data Path

Client ↔ DataNodes

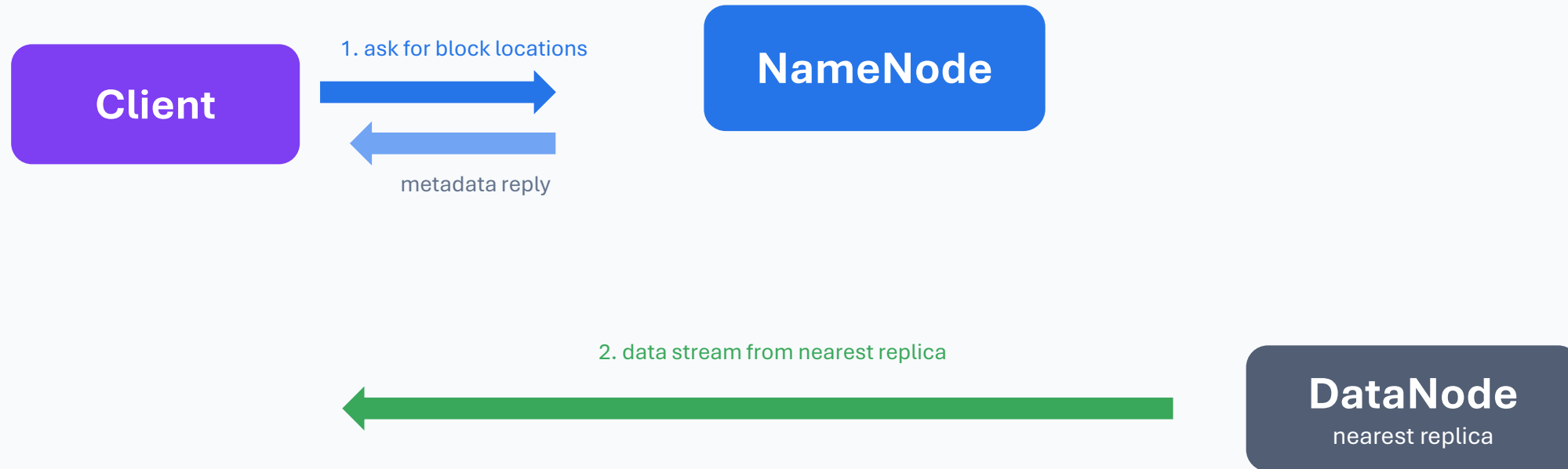
Actual bytes

Read / write

Replication pipeline

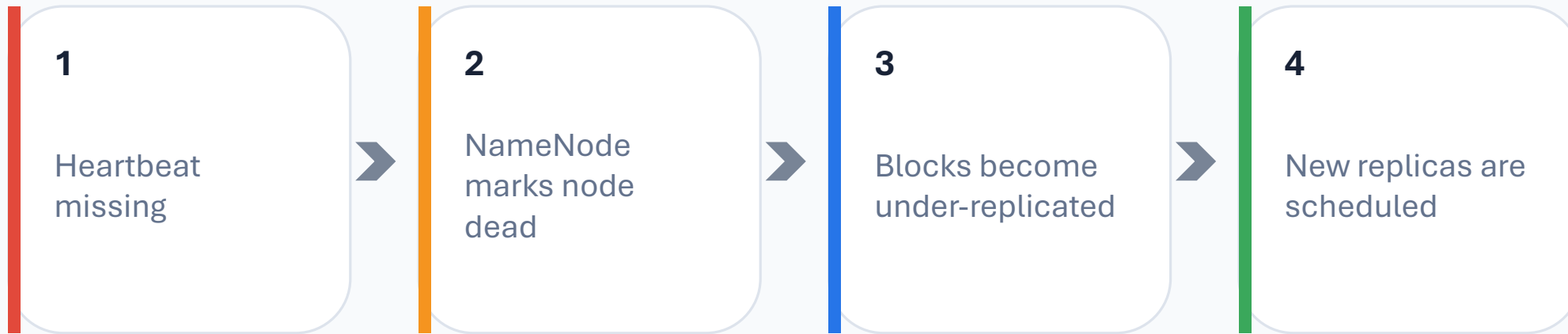
The NameNode does NOT carry user data.

How HDFS Reads a File



Le client obtient les localisations auprès du NameNode, puis lit directement une réplique appropriée.

What Happens When a Node Fails?



La réplication est restaurée automatiquement lorsque des blocs deviennent sous-répliqués.

HDFS Loves Large Files

Good Fit

Large files
Sequential / streaming access
High throughput
Write once, read many

Poorer Fit

Tiny files at huge scale
Low-latency random updates
Frequent in-place modifications

HDFS a été conçu pour de grands datasets et des lectures à débit élevé.

The Small Files Problem

10,000,000 small files

Chaque fichier ajoute des entrées de namespace et, souvent, des blocs à suivre.

NameNode metadata

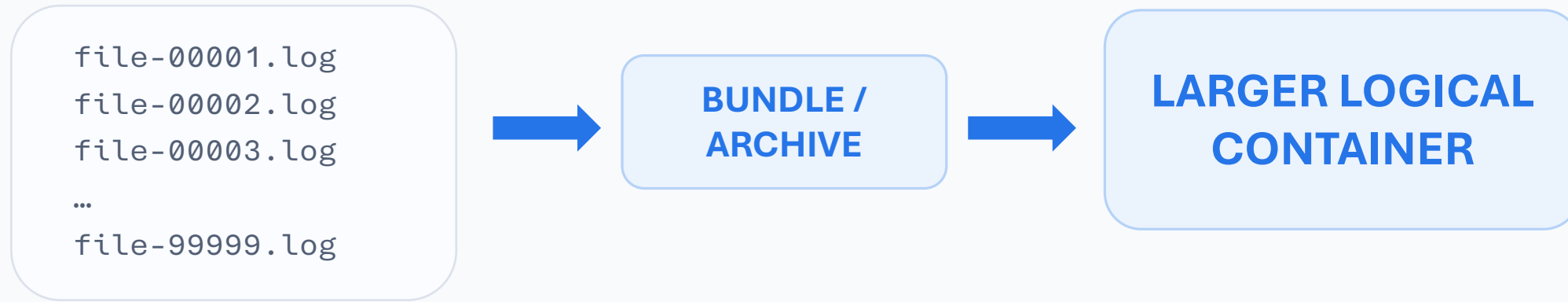
Namespace conservé en mémoire pour servir rapidement les opérations.

Memory pressure

Le nombre de fichiers devient le problème, même si chaque fichier est minuscule.

Le coût principal vient du très grand nombre d'entrées de namespace/blocs à gérer, pas uniquement des octets stockés.

How Can We Solve It?



Question : peut-on regrouper les petits fichiers tout en conservant un accès logique individuel ?

Hadoop Archives - HAR

Une réponse historique au small files problem

Many small files

hadoop archive

logs.har

Still addressable

HAR exposes a filesystem-like view

Archive is immutable after creation

HAR regroupe des petits fichiers dans une structure d'archive avec index et fichiers part-*.

How HAR Works

1 · Archive

logs.har

_masterindex

_index

part-0

part-1

...

2 · Index

logical filename

→ location inside part files

3 · Logical Access

har:///.../logs.har/...

Files remain individually addressable through the HAR filesystem layer.

La création d'une archive HAR est historiquement un job MapReduce ; les archives sont immuables.

HAR: Benefits & Limitations

Benefits

- Reduce namespace pressure
- Keep logical file access
- Native Hadoop concept
- Useful for immutable archives

Limitations

- Immutable archive
- Creation has a cost
- Added lookup complexity
- Not a universal modern solution

HAR est une solution à connaître, pas une réponse automatique à tous les pipelines modernes.

HDFS Solved a Huge Problem - But Has Constraints

Metadata center

NameNode remains operationally important

Cluster operations

Servers, disks, balancing, upgrades

Compute + Storage

Historically coupled in the cluster

Capacity planning

You operate the storage infrastructure

What If Storage Was a Service?

FROM

Operate Storage

Servers
Disks
DataNodes
Replication
Capacity planning

TO

Consume Storage

Applications use an API
Infrastructure is abstracted
Capacity is managed as a service

Distributed filesystem opéré par l'équipe vers le stockage objet managé.

Object Storage

Bucket + Object + Key

BUCKET dora-cold-storage

OBJECT
logs/2026/01/app-01.gz

OBJECT
logs/2026/01/app-02.gz

OBJECT
audit/manifest.json

Object Storage n'est pas HDFS dans le cloud : le modèle de données et les opérations sont différents.

HDFS vs Object Storage

HDFS

Distributed filesystem
Blocks
NameNode / DataNodes
Cluster operated
Strong filesystem semantics

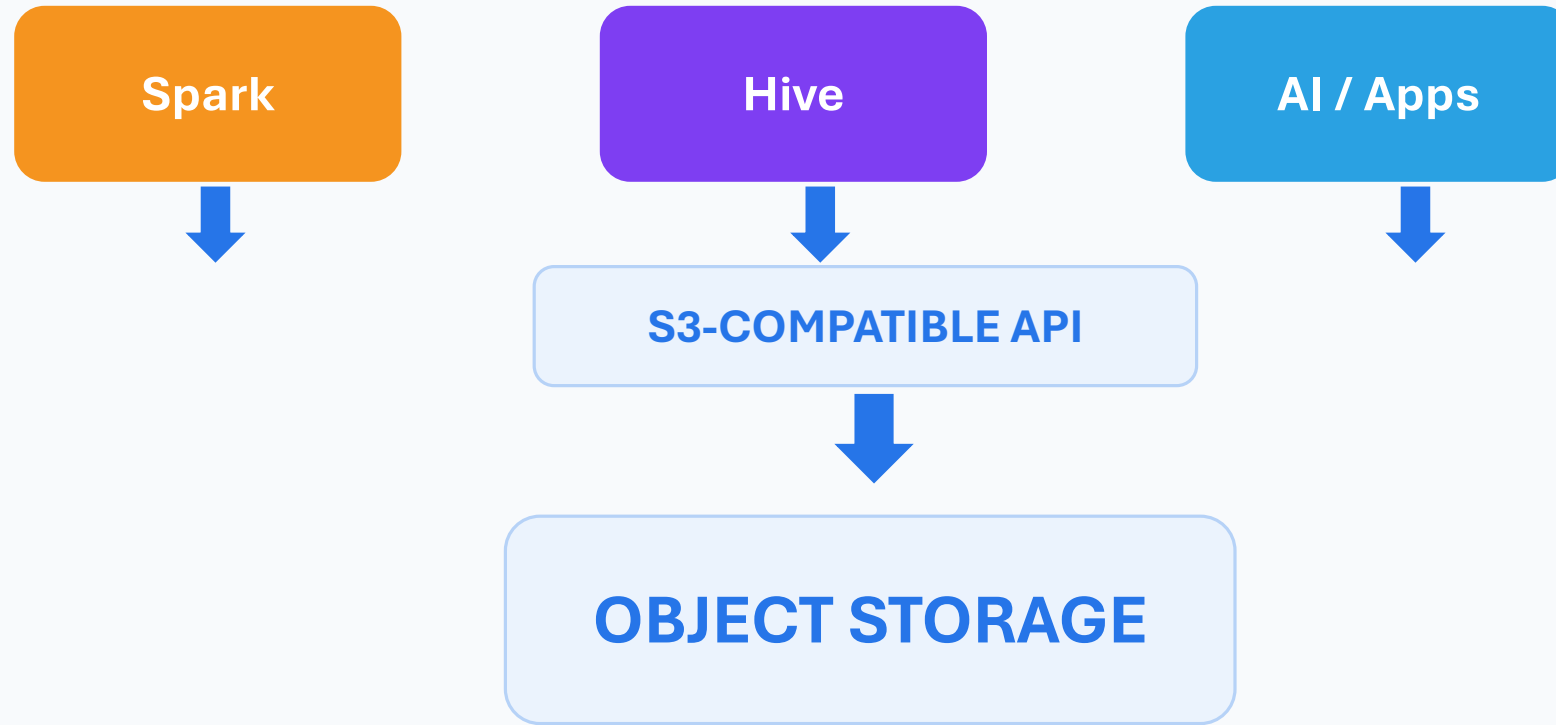
Object Storage

Object API
Objects + metadata
Bucket / key
Infrastructure abstracted
Different filesystem semantics

Same broad goal: massive durable storage. Different architecture and semantics.

The S3 Model

Decouple compute from storage



Le calcul et le stockage sont découplés : plusieurs moteurs accèdent au même stockage objet via la même API.

Why Modern Data Platforms Use Object Storage

Decoupling

Scale compute and storage independently

Elasticity

Storage capacity without adding HDFS nodes

Managed Durability

Provider operates redundancy mechanisms

Lifecycle

Move data across storage classes over time

HDFS Is Not Dead

Architecture is about trade-offs

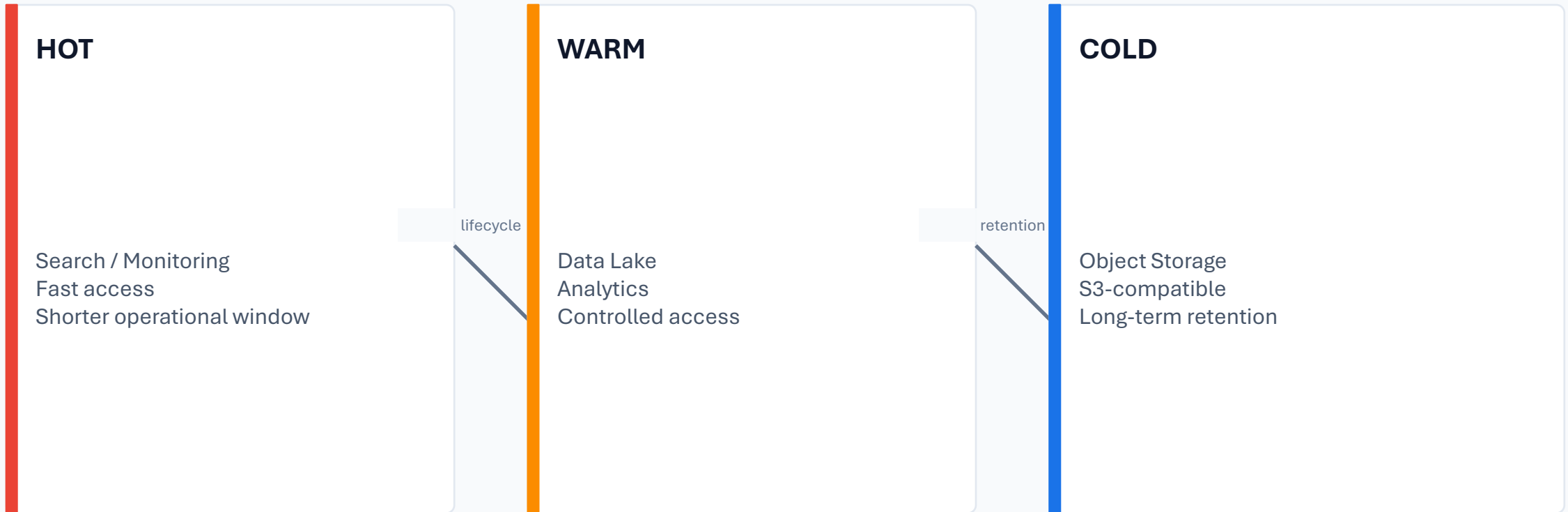
HDFS

- Great model for distributed storage
- Data locality
- Controlled/on-prem clusters
- High-throughput Hadoop workloads

Object Storage

- Cloud-native data lakes
- Compute/storage separation
- Managed capacity
- Lifecycle & cold tiers

DORA - Hot → Warm → Cold



Pour le projet : le stockage à froid devient un choix d'architecture distinct du monitoring live.

Scaleway Object Storage

A concrete S3-compatible cold-storage option

S3-compatible

Use common S3 clients and APIs

Lifecycle

Standard → colder storage classes

Archive

Glacier class for long-term archived data

HDFS pour le stockage distribué, Object Storage pour les architectures cloud modernes.

Key Takeaways

1

HDFS distributes files as blocks across DataNodes

2

NameNode manages metadata; DataNodes store bytes

3

Replication + Rack Awareness provide resilience

4

Small files stress namespace metadata; HAR is a historical mitigation

5

Object Storage changes the model by decoupling compute and storage

Now You Operate HDFS

TP 02 - chargement, administration et réplication

hdfs dfs

filesystem operations

fsck

blocks & locations

setrep

replication factor

dfsadmin

cluster health

Puis : construire les zones raw / archive / processed / audit du Data Lake DORA.